

Báo cáo bài tập EX12: Training neural network: Learning rate, dropout, activation function

```
In [1]: import warnings
warnings.filterwarnings('ignore')

import numpy as np
import matplotlib.pyplot as plt
import cv2

import torch
import os
from torch import nn
import torch.nn.functional as F
from torch.utils.data import Dataset
from torch.utils.data import DataLoader
from torchvision import datasets
from torchvision.transforms import ToTensor

from torch.utils.tensorboard import SummaryWriter

%load_ext tensorboard
```

```
In [2]: train_data = datasets.MNIST(
    root = 'data',
    train = True,
    download = True
)
test_data = datasets.MNIST(
    root = 'data',
    train = False
)
(x_train, y_train) = train_data.data[:].detach().numpy(), train_data.targets[:].detach().numpy()
(x_test, y_test) = test_data.data[:].detach().numpy(), test_data.targets[:].detach().numpy()
print('Training image: ', x_train.shape)
print('Testing image: ', x_test.shape)
print('Training label: ', y_train.shape)
print('Testing label: ', y_test.shape)

Training image: (60000, 28, 28)
Testing image: (10000, 28, 28)
Training label: (60000,)
Testing label: (10000,)
```

Để dễ hình dung về dữ liệu, có thể sử dụng thư viện matplotlib quan sát một vài mẫu dữ liệu:

```
In [3]: for i in range(30):
        idx = np.random.randint(0, x_train.shape[0])
        image = x_train[idx]
        plt.subplot(3, 10, i + 1), plt.imshow(image, cmap='gray')
        plt.title(y_train[idx]), plt.xticks([]), plt.yticks([])
    plt.show()
```



```
In [4]: batch_size = 64
        num_classes = 10
        epochs = 20
        # input image dimensions
        img_rows, img_cols = 28, 28

        x_train = x_train.reshape(x_train.shape[0], 1, img_rows, img_cols)
        x_test = x_test.reshape(x_test.shape[0], 1, img_rows, img_cols)
        input_shape = (1, img_rows, img_cols)
```

Trước khi đưa vào mô hình, cần chuẩn hoá giá trị điểm ảnh về trong khoảng [0,1] nhằm giúp các thuật toán tối ưu hội tụ nhanh hơn:

```
In [5]: x_train = x_train.astype('float32')
        x_test = x_test.astype('float32')
        x_train /= 255
        x_test /= 255
```

```
In [6]: print(x_train.shape, y_train.shape)
```

```
(60000, 1, 28, 28) (60000,)
```

Phần 2: Xây dựng mô hình phân loại

Trong phần này, chúng ta sẽ xây dựng một mô hình phân loại đơn giản cho bài toán nhận diện chữ số viết tay sử dụng thư viện keras. Từ thử nghiệm tác động của các yếu tố như learning rate, dropout, activation function đến quá trình huấn luyện mạng.

```
In [7]: class DeepModel(nn.Module):
    def __init__(self, dropout_rate):
        super(DeepModel, self).__init__()
        self.conv_1 = nn.Conv2d(1, 32, kernel_size = 3, stride = (1, 1), padding=0, bias=False, dilation=1)
        self.conv_2 = nn.Conv2d(32, 64, kernel_size = 3, stride = (1, 1), padding=0, bias=False, dilation=1)

        self.dropout = nn.Dropout(dropout_rate)

        self.dense_1 = nn.Linear(12*12*64, 512)
        self.dense_2 = nn.Linear(512, 10)

    def forward(self, x):
        y=F.relu(self.conv_1(x))
        y=F.relu(self.conv_2(y))
        y=F.max_pool2d(y, 2)
        y=self.dropout(y)
        y=torch.flatten(y, 1)
        y = F.relu(self.dense_1(y))
        y=self.dense_2(y)

        return F.log_softmax(y, dim=1)
```

Kiểm tra số lượng tham số

```
In [8]: # Tự động nhận diện thiết bị
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
simple_model = DeepModel(dropout_rate = 0.5).to(device)
for param in simple_model.parameters():
    if param.requires_grad:
        print('param autograd')
        break

input = torch.rand(2, 1, 28, 28).to(device)
output = simple_model(input) # type: torch.Tensor

model_parameters = filter(lambda p: p.requires_grad, simple_model.parameters())
params = sum([np.prod(p.size()) for p in model_parameters])
print('Number of parameter:', params)

param autograd
Number of parameter: 4742954
```

Khởi tạo Generator

```
In [9]: class Generator(Dataset):
        def __init__(self, images, labels):
            self.images = images
            self.labels = labels

        def __len__(self):
            return self.images.shape[0]

        def __getitem__(self, idx):
            return self.images[idx], self.labels[idx]
```

```
In [10]: training_data = Generator(x_train, y_train)
         train_dataloader = DataLoader(training_data, batch_size=32, shuffle=True)
```

```
In [11]: test_data = Generator(x_test, y_test)
         test_dataloader = DataLoader(test_data, batch_size=32, shuffle=True)
```

- Dropout rate = 0

```
In [12]: use_cuda = torch.cuda.is_available() #GPU cuda
         best_loss = float('inf')

         model = DeepModel(dropout_rate = 0)

         optimizer = torch.optim.Adam(model.parameters())
         if use_cuda:
             model = torch.nn.parallel.DataParallel(model.cuda()) # , device_ids=[0, 1, 2, 3]
             torch.backends.cudnn.benchmark = True
```

```
In [13]: def train(model, epoch, writer):
         print('\n ##### Train phase, Epoch: {} #####'.format(epoch))
         model.train()
         train_loss = 0
         running_loss = 0
         print('\n Learning rate at this epoch is: ', optimizer.param_groups[0]['lr'], '\n')
         for (batch_idx, target_tuple) in enumerate(train_dataloader):
             if use_cuda:
                 target_tuple = [target_tensor.cuda(non_blocking=True) for target_tensor in target_tuple]

             images, labels = target_tuple
             # Convert label to long type pytorch
             labels = torch.tensor(labels, dtype=torch.long)

             optimizer.zero_grad() # zero the gradient buff
             output_tuple = model(images)

             loss = F.nll_loss(output_tuple, labels)

             loss.backward() # retain_graph=True
             optimizer.step()

             train_loss += loss.item() # Loss
             running_loss += loss.item()
             if batch_idx % 50 == 49:
                 writer.add_scalar('training loss', running_loss/50, epoch * len(train_dataloader) + batch_idx)
                 running_loss = 0
             #print('##### Epoch:', epoch, ', -- batch:', batch_idx, '/', len(train_dataloader), ',
             #      'Train Loss: %.3f, accumulated average loss: %.3f #####' % (loss.item(), train_l
```

```
In [14]: def test(model, epoch, writer):
    print('\n ##### Test phase, Epoch: {} #####'.format(epoch))
    model.eval()
    with torch.no_grad():
        test_loss = 0
        correct = 0
        for (batch_idx, target_tuple) in enumerate(test_dataloader):
            if use_cuda:
                target_tuple = [target_tensor.cuda(non_blocking=True) for target_tensor in target_tuple]

            images, labels = target_tuple
            # Convert Label to long type pytorch
            labels = torch.tensor(labels, dtype=torch.long)
            output_tuple = model(images)
            #print(output_tuple.shape)

            _, predicted = torch.max(output_tuple.data, 1)
            correct += (predicted == labels).sum().item()

            loss = F.nll_loss(output_tuple, labels)

            test_loss += loss.item() # Loss
            #print('##### Epoch:', epoch, ', -- batch:', batch_idx, '/', len(test_dataloader), ',
            #      'Test Loss: %.3f, accumulated average Loss: %.3f #####' % (loss.item(), test_lo
        acc = correct*100/len(test_data)
        print('Accuracy: ', acc)
        writer.add_scalar('test accuracy', acc, epoch)
```

```
In [15]: def train_and_test(model, epoch_num = 5, summary_path='runs/mnist_experiment_dropout'):
    writer = SummaryWriter(summary_path)
    for epoch in range(epoch_num):
        train(model, epoch, writer)
        test(model, epoch, writer)
```

Huấn luyện mô hình

```
In [16]: train_and_test(model, 1, 'runs/mnist_experiment_dropout=0')
```

```
##### Train phase, Epoch: 0 #####
```

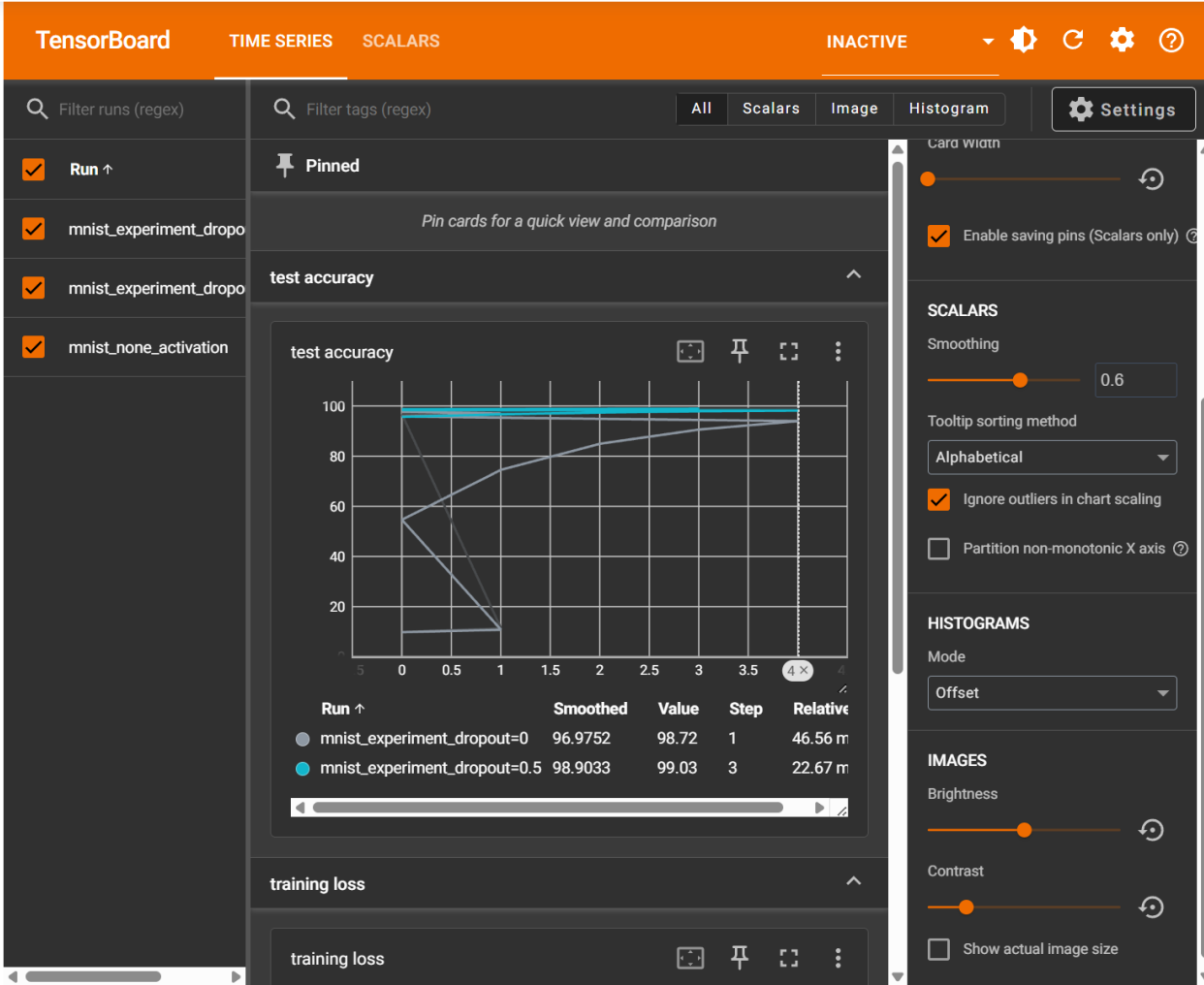
Learning rate at this epoch is: 0.001

```
##### Test phase, Epoch: 0 #####
```

Accuracy: 98.79

```
In [29]: from tensorboard import program
```

```
# Thử tìm các tiến trình cũ và đóng chúng bằng code Python thay vì lệnh hệ thống
%load_ext tensorboard
%tensorboard --logdir=runs --port=6007
```



- Dropout rate = 0.5

```
In [18]: use_cuda = torch.cuda.is_available() #GPU cuda
best_loss = float('inf')

model=DeepModel(dropout_rate=0.5)

optimizer = torch.optim.Adam(model.parameters())
if use_cuda:
    model = torch.nn.parallel.DataParallel(model.cuda()) # , device_ids=[0, 1, 2, 3]
    torch.backends.cudnn.benchmark = True
```

```
In [19]: train_and_test(model, 1, 'runs/mnist_experiment_dropout=0.5')
```

```
##### Train phase, Epoch: 0 #####
```

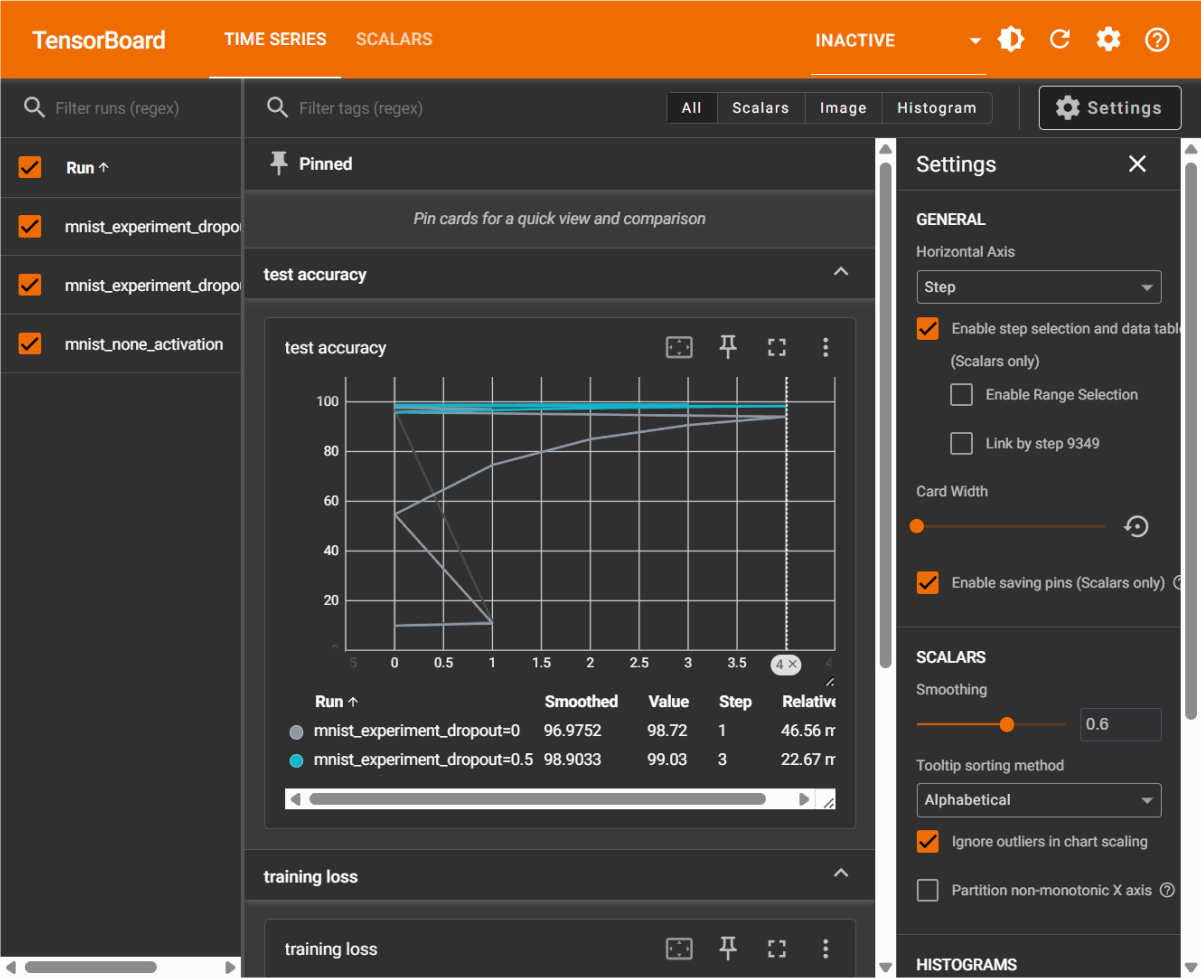
```
Learning rate at this epoch is: 0.001
```

```
##### Test phase, Epoch: 0 #####
```

```
Accuracy: 98.41
```

```
In [30]: from tensorboard import program

# Thử tìm các tiến trình cũ và đóng chúng bằng code Python thay vì Lệnh hệ thống
%load_ext tensorboard
%tensorboard --logdir=runs --port=6007
```



Phần 2.2: Vai trò của activation function

Viết mô hình đơn giản cho phép nhận các loại hàm kích hoạt khác nhau

```
In [46]: class SimpleModel(nn.Module):
    def __init__(self, dropout_rate=0.5, activation = lambda x: x):
        super(SimpleModel, self).__init__()
        self.dense_1 = nn.Linear(28*28, 512)
        self.dropout = nn.Dropout(dropout_rate)
        self.dense_2 = nn.Linear(512, 10)
        self.activation = activation

    def forward(self, x):
        y=torch.flatten(x, 1)
        y=self.activation(self.dense_1(y))
        y=self.dropout(y)
        y=self.dense_2(y)
        return y
```

Kiểm tra số lượng tham số

```
In [47]: simple_model = SimpleModel()
for param in simple_model.parameters():
    if param.requires_grad:
        print('param autograd')
        break

input = torch.rand(1, 28, 28)
output = simple_model(input) # type: torch.Tensor

model_parameters = filter(lambda p: p.requires_grad, simple_model.parameters())
params = sum([np.prod(p.size()) for p in model_parameters])
print('Number of parameter:', params)
```

```
param autograd
Number of parameter: 407050
```

trước tiên hãy thử xem, sẽ thế nào nếu không sử dụng hàm kích hoạt cho lớp ẩn.

In [42]:

```
simple_model = SimpleModel(dropout_rate=0.5, activation=lambda x: x)

# Đùng quên đẩy lên GPU RTX 4060 của bạn
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
simple_model.to(device)

optimizer = torch.optim.Adam(simple_model.parameters())

# Chạy lại huấn luyện
train_and_test(simple_model, 1, 'runs/mnist_none_activation')

##### Train phase, Epoch: 0 #####

Learning rate at this epoch is: 0.001

##### Test phase, Epoch: 0 #####
Accuracy: 11.35
```

Kết quả đạt được là tương đối tệ trên tập dữ liệu MNIST. Tiếp theo chúng ta sẽ thử nghiệm và so sánh kết quả khi sử dụng

In [43]:

```
for activation in [nn.Sigmoid(), nn.Tanh(), nn.ReLU()]:
    simple_model=SimpleModel(dropout_rate=0.5, activation=activation)
    train_and_test(simple_model, 1, 'runs/mnist_' + str(activation))

##### Train phase, Epoch: 0 #####

Learning rate at this epoch is: 0.001

##### Test phase, Epoch: 0 #####
Accuracy: 8.92

##### Train phase, Epoch: 0 #####

Learning rate at this epoch is: 0.001

##### Test phase, Epoch: 0 #####
Accuracy: 9.71

##### Train phase, Epoch: 0 #####

Learning rate at this epoch is: 0.001

##### Test phase, Epoch: 0 #####
Accuracy: 5.15
```

```
In [45]: class SimpleModel(nn.Module):
def __init__(self, dropout_rate=0.5, activation=nn.Sigmoid()):
    super(SimpleModel, self).__init__()
    self.dense_1 = nn.Linear(28*28, 512)
    self.dropout = nn.Dropout(dropout_rate)
    self.dense_2 = nn.Linear(512, 10)
    self.activation = activation

def forward(self, x):
    y=torch.flatten(x, 1)
    y=self.activation(self.dense_1(y))
    y=self.dropout(y)
    y=self.dense_2(y)
    return y

simple_model=SimpleModel(dropout_rate=0.5, activation=nn.Sigmoid())
train_and_test(simple_model, 1, 'runs/mnist_' + str(nn.Sigmoid()))
```

```
##### Train phase, Epoch: 0 #####
```

```
Learning rate at this epoch is: 0.001
```

```
##### Test phase, Epoch: 0 #####
```

```
Accuracy: 9.75
```

Phần 2.3: Vai trò của hệ số học learning rate

Ta tiếp tục quan sát tác động của learning rate đến quá trình học của mạng:

```
In [61]: from torch.utils.tensorboard import SummaryWriter

def train_and_test(model, epochs, log_dir, learning_rate):
    # Xác định thiết bị
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model.to(device)

    # Khởi tạo Optimizer và Scheduler dựa trên learning_rate truyền vào
    optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)

    # Scheduler này sẽ giữ nguyên LR nếu bạn không muốn giảm (gamma=1.0)
    scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=1, gamma=1.0)

    # Khởi tạo Writer để lưu log vào đường dẫn riêng
    writer = SummaryWriter(log_dir)

    for epoch in range(1, epochs + 1):
        # Gọi hàm train cũ của bạn
        train(model, epoch, writer)

    writer.close()
    print(f"--- Xong thử nghiệm cho LR: {learning_rate} ---")
learning_rates = [1E-0, 1E-1, 1E-2, 1E-3, 1E-4, 1E-5, 1E-6, 1E-7]
for lr in learning_rates:
    print(f"--- Đang thử nghiệm với Learning Rate: {lr} ---")

    # 1. Khởi tạo lại mô hình để đảm bảo tính công bằng (không học tiếp từ LR trước)
    model = SimpleModel(dropout_rate=0.5, activation=nn.ReLU())

    # 2. Định nghĩa đường dẫn log cho TensorBoard để so sánh các đường cong
    log_dir = f'runs/mnist_lr_{lr}'

    # 3. Gọi hàm huấn luyện (giả định hàm này nhận tham số lr)
    # Nếu hàm train_and_test của bạn chưa nhận lr, bạn cần truyền nó vào Optimizer bên trong hàm đó
    train_and_test(model, 1, log_dir, learning_rate=lr)

--- Đang thử nghiệm với Learning Rate: 1.0 ---

##### Train phase, Epoch: 1 #####
```

```
##### Train phase, Epoch: 1 #####
Learning rate at this epoch is: 0.001
--- Xong thử nghiệm cho LR: 1.0 ---
--- Đang thử nghiệm với Learning Rate: 0.1 ---

##### Train phase, Epoch: 1 #####
Learning rate at this epoch is: 0.001
--- Xong thử nghiệm cho LR: 0.1 ---
--- Đang thử nghiệm với Learning Rate: 0.01 ---

##### Train phase, Epoch: 1 #####
Learning rate at this epoch is: 0.001
--- Xong thử nghiệm cho LR: 0.01 ---
--- Đang thử nghiệm với Learning Rate: 0.001 ---

##### Train phase, Epoch: 1 #####
Learning rate at this epoch is: 0.001
--- Xong thử nghiệm cho LR: 0.001 ---
--- Đang thử nghiệm với Learning Rate: 0.0001 ---

##### Train phase, Epoch: 1 #####
Learning rate at this epoch is: 0.001
--- Xong thử nghiệm cho LR: 0.0001 ---
--- Đang thử nghiệm với Learning Rate: 1e-05 ---

##### Train phase, Epoch: 1 #####
Learning rate at this epoch is: 0.001
--- Xong thử nghiệm cho LR: 1e-05 ---
--- Đang thử nghiệm với Learning Rate: 1e-06 ---

##### Train phase, Epoch: 1 #####
Learning rate at this epoch is: 0.001
--- Xong thử nghiệm cho LR: 1e-06 ---
--- Đang thử nghiệm với Learning Rate: 1e-07 ---

##### Train phase, Epoch: 1 #####
Learning rate at this epoch is: 0.001
```

- Giảm learning rate theo cơ chế:

$l_{nr} = \text{init_l}_{nr} / (1 + \text{decay} * \text{step})$

```
In [69]: def train(model, optimizer, scheduler, epoch, writer):
    print('\n ##### Train phase, Epoch: {} #####'.format(epoch))
    model.train()
    train_loss = 0
    running_loss = 0
    print('\n Learning rate at this epoch is: ', scheduler.get_last_lr(), '\n')
    for (batch_idx, target_tuple) in enumerate(train_dataloader):
        if use_cuda:
            target_tuple = [target_tensor.cuda(non_blocking=True) for target_tensor in target_tuple]

        images, labels = target_tuple
        # Convert label to long type pytorch
        labels = torch.tensor(labels, dtype=torch.long)

        optimizer.zero_grad() # zero the gradient buff
        output_tuple = model(images)

        loss = F.nll_loss(output_tuple, labels)

        loss.backward() # retain_graph=True
        optimizer.step()

        train_loss += loss.item() # loss
        running_loss += loss.item()
        if batch_idx % 50 == 49:
            writer.add_scalar('training loss', running_loss/50, epoch * len(train_dataloader) + batch_idx)
            running_loss = 0
            # print('##### Epoch: ', epoch, ', -- batch: ', batch_idx, '/', len(train_dataloader),
            #       'Train Loss: %.3f, accumulated average loss: %.3f #####' % (loss.item(),
            scheduler.step()
```

```
In [70]: def train_and_test(model, optimizer, scheduler, epoch_num = 5, summary_path='runs/mnist_experiment_dropout'):
    writer = SummaryWriter(summary_path)
    for epoch in range(epoch_num):
        train(model, optimizer, scheduler, epoch, writer)
        test(model, epoch, writer)
```

Viết đoạn code cho phép huấn luyện mô hình với tốc độ học giảm dần qua từng epoch với hệ số 0.9

```
In [71]: # 1. Chọn bộ tối ưu hóa (Optimizer)
optimizer = torch.optim.Adam(simple_model.parameters(), lr=0.01)

# 2. Thiết lập cách giảm LR: mỗi epoch nhân với 0.9
scheduler = torch.optim.lr_scheduler.ExponentialLR(optimizer, gamma=0.9)

# 3. Chạy hàm huấn luyện
train_and_test(simple_model, optimizer, scheduler, 20, 'runs/exponentialLR')
```

```
##### Train phase, Epoch: 0 #####

Learning rate at this epoch is: [0.01]

##### Test phase, Epoch: 0 #####
Accuracy: 11.35

##### Train phase, Epoch: 1 #####

Learning rate at this epoch is: [0.009000000000000001]

##### Test phase, Epoch: 1 #####
Accuracy: 11.35

##### Train phase, Epoch: 2 #####
```